

Python Lists & Tuples

A small quick-reference guide

1. Python Lists

A list is an ordered and mutable collection. It can contain different data types and allows duplicate values.

Example:

```
numbers = [10, 20, 30, 40]
names = ["Alice", "Bob", "Alice"]
```

2. Accessing List Elements

Python uses zero-based indexing. Negative indexing starts from the last element.

```
numbers = [10, 20, 30, 40]
print(numbers[0]) # 10
print(numbers[-1]) # 40
```

3. Common List Operations

Operation	Example	Purpose
append()	a.append(5)	Add an item at the end
insert()	a.insert(1, 5)	Insert at an index
remove()	a.remove(5)	Remove a value
pop()	a.pop()	Remove and return an item
sort()	a.sort()	Sort the list
reverse()	a.reverse()	Reverse the list
len()	len(a)	Get number of items

4. List Slicing

Slicing extracts part of a list using the form list[start:stop:step]. The stop index is excluded.

```
a = [0, 1, 2, 3, 4, 5]
print(a[1:4]) # [1, 2, 3]
print(a[:3]) # [0, 1, 2]
print(a[::2]) # [0, 2, 4]
```

5. Python Tuples

A tuple is an ordered and immutable collection. Once created, its elements cannot normally be changed.

Example:

```
point = (10, 20)
colors = ("red", "green", "blue")
```

6. Accessing and Unpacking Tuples

Tuples support indexing and slicing just like lists. They can also be unpacked into separate variables.

```
point = (10, 20)
print(point[0]) # 10
```

```
x, y = point
print(x, y) # 10 20
```

7. List vs Tuple

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)
Mutable	Yes	No
Duplicates	Allowed	Allowed
Indexing	Supported	Supported
Typical use	Data that may change	Fixed/constant data

8. Important Points to Remember

- Lists are mutable: elements can be added, removed, or changed.
- Tuples are immutable after creation.
- Both support indexing, slicing, iteration, and duplicate values.
- Use a list when the collection needs modification; use a tuple when the data should remain fixed.